

LegalEase

AI-Powered Legal Document Generator

Phase 5: Project Development

Project Report

1. Purpose of This Phase

The Development phase covers the actual implementation of LegalEase based on the design and plan established earlier — building the frontend, backend, AI generation engine, and export functionality into a working application.

2. Development Approach

Development is carried out sprint-wise, with each sprint delivering a demonstrable increment of functionality. Frontend and backend teams work in parallel against an agreed API contract, while the AI/NLP module is developed and tested independently before integration.

3. Technology Stack Used

Component	Technology / Tool
Frontend	React.js with component-based UI, Tailwind CSS for styling
Backend / API	Python (FastAPI) or Node.js (Express) exposing REST endpoints
AI / NLP Engine	LLM-based text generation combined with rule-based template filling
Database	PostgreSQL for structured data (users, templates, documents)
Authentication	JWT-based token authentication
Document Export	ReportLab / python-docx for PDF and Word generation
Version Control	Git with a shared repository (e.g., GitHub)

4. Module-wise Implementation Summary

4.1 Authentication Module

Implements user sign-up, login, password hashing, and session/token issuance so that only authenticated users can generate and access their documents.

4.2 Template & Clause Manager

Stores base structures for each supported document category along with a library of mandatory and optional clauses, allowing the AI engine to assemble documents consistently.

4.3 Input Processor

Accepts the user's natural-language description, extracts key entities (parties, dates, amounts, location, terms) and maps them to the corresponding template fields.

4.4 AI Generation Engine

Combines the extracted fields with the base template and calls the language model to draft, expand, or rephrase clauses in clear legal language, producing a coherent first draft.

4.5 Clause Validator

Cross-checks the generated draft against the mandatory clause list for the selected document type and flags any missing or incomplete sections for the user's attention.

4.6 Editor Module

Provides an in-browser rich-text editor so users can modify wording, add clauses, or remove sections before finalizing the document.

4.7 Export Module

Converts the finalized document into downloadable PDF and Word formats while preserving formatting such as headings, numbered clauses, and signature blocks.

4.8 History Module

Persists each generated document against the user's account, allowing retrieval, re-editing, or re-download at a later time.

5. Sample Workflow (Pseudocode)

```
input = get_user_description()
fields = extract_fields(input)
template = load_template(selected_category)
draft = ai_generate(template, fields)
validation = validate_clauses(draft, template.mandatory_clauses)
return draft, validation
```

6. Coding Standards Followed

- Consistent naming conventions across frontend and backend code.
- Modular file structure separating routes, business logic, and data access.
- Inline comments and docstrings for non-trivial functions, especially in the AI processing layer.
- Environment variables used for sensitive configuration (API keys, database credentials).

7. Version Control Practice

All code changes are tracked using Git, with feature branches created per module (e.g., feature/auth, feature/ai-engine, feature/export) and merged into the main branch after review, keeping the development history traceable.

8. Challenges Encountered During Development

- Ensuring AI-generated text remained consistent with the formal tone expected in legal documents.
- Mapping free-form natural language input reliably to structured template fields.
- Balancing AI-generated flexibility with the need for mandatory, non-negotiable clauses.
- Formatting exported PDF/Word documents to look professional and print-ready.

9. Integration Details

The frontend communicates with the backend exclusively through documented REST API endpoints, using JSON as the data exchange format. The AI Generation Engine is integrated as an internal service called by the backend, keeping the API keys and model configuration isolated from the client side for security.

10. Deployment Process

- 1 Build and package the frontend application for production.
- 2 Deploy the backend API and database to the chosen cloud hosting environment.
- 3 Configure environment variables for database connection, authentication secrets, and AI API keys.
- 4 Run a smoke test on the deployed environment to confirm core functionality works as expected.
- 5 Point the domain/URL to the deployed application for access during testing and demonstration.

11. Code Review Practice

Before merging into the main branch, each pull request is reviewed by at least one other team member for correctness, readability, and adherence to the agreed coding standards, with special attention given to the AI Generation Engine and Clause Validator given their central role in output quality.

12. Outcome of This Phase

By the end of this phase, a working end-to-end version of LegalEase is implemented and deployed: users can log in, describe a requirement, receive an AI-generated draft, edit it, and export a finished document — ready to proceed into formal testing.